

Understanding the HPT-QRC Paper

A Complete Concept-by-Concept Teaching Guide
From Time-Series Forecasting to Quantum Photonics

Prepared for the HPT-QRC research team

May 2026

Contents

I	Time Series & Statistics	4
1	AutoRegressive Models — AR(1) and AR(3)	4
2	HAR Model — Heterogeneous AutoRegressive	5
3	Realized Volatility (RV)	6
4	Ridge Regression	6
5	MSE and QLIKE Loss Functions	7
II	Machine Learning Foundations	9
6	Reservoir Computing and Echo State Networks (ESN)	9
7	Random Fourier Features (RFF)	10
8	Optuna and the TPE Algorithm	11
9	Walk-Forward Validation	12
III	Statistical Inference	13
10	The Diebold-Mariano (DM) Test	13
11	Newey-West HAC Variance	14
12	Hansen Model Confidence Set (MCS)	15
IV	Quantum Photonics	17

13 Quantum States and Fock Space	17
14 Linear Optical Interferometers and Phase Encoding	18
15 The Permanent of a Matrix and Boson Sampling	19
16 Perceval, SLOS, and MerLin	19
17 LexGrouping — Compressing Fock Probabilities	21
 V Benchmarks and Architecture	 22
18 NARMA-10 Benchmark	22
19 Mackey-Glass Benchmark	23
20 Information Processing Capacity (IPC)	23
21 Virtual Depth	25
22 The Fock-Space Scaling and Heterogeneous Ensemble	25

How to Use This Guide

This guide is written like a textbook chapter. Every technical term used in the paper “Heterogeneous Photon-Number Ensembles in Linear-Optical Quantum Reservoir Computing for Chaotic and Volatility Forecasting” is explained from first principles. You do not need a physics or statistics degree to follow along — just read in order. Each concept builds on the previous ones.

The guide is divided into five parts:

1. **Time Series & Statistics** — AR, HAR, RV, MSE, QLIKE, Ridge
2. **Machine Learning Foundations** — ESN, RC, RFF, Optuna TPE, Walk-Forward
3. **Statistical Inference** — DM test, HAC variance, Andrews bandwidth, MCS
4. **Quantum Photonics** — Fock states, interferometers, SLOS, LexGrouping
5. **Benchmarks & Architecture** — NARMA-10, Mackey-Glass, IPC, Virtual Depth

Part I

Time Series & Statistics

1 AutoRegressive Models — AR(1) and AR(3)

What is an AutoRegressive Model?

Imagine you are trying to predict tomorrow's temperature. A simple strategy is: "tomorrow's temperature is likely close to today's, plus some random variation." That is exactly what an AutoRegressive (AR) model does — it predicts the next value of a time series using a weighted sum of its own past values.

The "order" p of an AR model (written $\text{AR}(p)$) is simply *how many past values* you look back at. $\text{AR}(1)$ looks 1 step back; $\text{AR}(3)$ looks 3 steps back.

Math: $\text{AR}(p)$ Formula

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$$

- y_t : the value we want to predict at time t
- c : a constant intercept (like an average level)
- ϕ_i : the *weight* given to the i -th past value (estimated from data by Ordinary Least Squares)
- ε_t : random noise — unpredictable part

For **AR(1)**: $y_t = c + \phi_1 y_{t-1} + \varepsilon_t$

For **AR(3)**: $y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \phi_3 y_{t-3} + \varepsilon_t$

Concrete Example: Volatility Persistence

Financial volatility is *persistent* — high-volatility days tend to be followed by more high-volatility days. For S&P 500 realized volatility, $\hat{\phi}_1 \approx 0.85$ is typical:

$$\widehat{RV}_{t+1} \approx 0.003 + 0.85 \times RV_t$$

So if today's RV is 0.02, we forecast tomorrow's as $0.003 + 0.85 \times 0.02 = 0.020$.

Key Takeaway: AR models are the *floor* for forecasting. Any model worth using should statistically beat them. In the paper they are the simplest baselines in all three benchmarks.

2 HAR Model — Heterogeneous AutoRegressive

The Key Idea — Multiple Horizons of Persistence

Financial markets are watched by many different types of participants. A day trader looks at what happened in the last few hours. A hedge fund looks at last week. A pension fund thinks in terms of months.

Corsi (2009) had a beautiful insight: *realized volatility is the sum of these overlapping, heterogeneous time horizons*. Rather than picking one lag, why not include *daily, weekly, and monthly* moving averages of past volatility simultaneously? That is the HAR model.

Despite having only 4 parameters (intercept + three coefficients), HAR is **notoriously hard to beat** — deep learning models with millions of parameters often cannot statistically outperform it on monthly RV data.

Math: HAR Formula

$$RV_{t+1} = c + \beta_d \underbrace{RV_t}_{\text{daily}} + \beta_w \underbrace{\overline{RV}_t^{(5)}}_{\text{5-day avg}} + \beta_m \underbrace{\overline{RV}_t^{(22)}}_{\text{22-day avg}} + \varepsilon_{t+1}$$

where:

$$\begin{aligned}\overline{RV}_t^{(5)} &= \frac{1}{5}(RV_t + RV_{t-1} + \dots + RV_{t-4}) \\ \overline{RV}_t^{(22)} &= \frac{1}{22}(RV_t + \dots + RV_{t-21})\end{aligned}$$

The 4 parameters $c, \beta_d, \beta_w, \beta_m$ are estimated by OLS.

Concrete Example: Why HAR Wins on Monthly Data

S&P 500 monthly realized volatility has a *long memory* pattern — today's volatility is correlated with volatility from many months ago. HAR approximates this long memory using a cascade of three short-memory components: the daily component captures recent spikes; the weekly component captures medium-term regimes; the monthly component captures structural slow-moving volatility levels. Together they explain $\approx 70\%$ of variance with only 4 parameters.

HARX (HAR with Exogenous Variables). HARX simply appends extra predictor variables $\mathbf{X}_t$ to the HAR formula:

$$RV_{t+1} = c + \beta_d RV_t + \beta_w \overline{RV}_t^{(5)} + \beta_m \overline{RV}_t^{(22)} + \gamma^\top \mathbf{X}_t + \varepsilon_{t+1}$$

Common choices for $\mathbf{X}_t$: VIX (implied volatility), jump components, bid-ask spreads. In the paper, Fama-French and macroeconomic factors are used as exogenous inputs.

Key Takeaway: HAR is the primary classical rival for the S&P 500 task. If HPT-QRC (tuned) is statistically indistinguishable from HAR, that is a very respectable result given HAR's strong track record.

3 Realized Volatility (RV)

What is Realized Volatility?

“Volatility” means how much asset prices fluctuate. Classical models (like GARCH) treat volatility as a *hidden* (latent) variable that must be inferred from daily close-to-close returns. Realized Volatility is different: it *measures* volatility directly from high-frequency (e.g., 5-minute) intraday data, treating it as if it were directly observable.

This transforms a hard latent-variable estimation problem into a simple regression problem: given today’s observable RV, predict tomorrow’s.

Math: RV Formula

Given M intraday log-returns $r_{t,j}$ over day t :

$$RV_t = \sum_{j=1}^M r_{t,j}^2, \quad r_{t,j} = \log\left(\frac{P_{t,j}}{P_{t,j-1}}\right)$$

For 5-minute returns over a 6.5-hour US trading day: $M = 78$.

As $M \rightarrow \infty$ (continuous sampling), by the theory of quadratic variation:

$$RV_t \xrightarrow{p} \int_0^1 \sigma_t^2 dt + \sum_j J_j^2$$

(integrated variance + squared jumps). So RV is a *model-free, consistent* estimator of total latent variance.

Key Takeaway: The S&P 500 RV dataset covers 1950–2017 at monthly frequency. Forecasting it well is the “real-world relevance” claim of the paper.

4 Ridge Regression

Why Not Just Use Ordinary Least Squares (OLS)?

OLS finds the weights $\hat{\beta}$ that minimize the squared prediction error. It works beautifully when you have many observations and few features. But when the number of features is large (say, 270 quantum features from 3 reservoirs $\times$ 3 virtual layers $\times$ 10 lex-out dimensions), OLS over-fits badly: the coefficients become enormous and noisy, performing well on training data but terribly on new data.

Ridge regression fixes this by adding a *penalty* on the size of the coefficients — it shrinks them toward zero, sacrificing a tiny bit of training accuracy for a huge gain in generalisation.

Math: Ridge Objective and Solution

Objective:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|y - X\beta\|^2 + \alpha \|\beta\|^2$$

Closed-form solution:

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \alpha I)^{-1} X^\top y$$

- $\alpha > 0$: regularisation strength (tuned via Optuna)
- αI : the identity times α added to $X^\top X$ makes the matrix invertible even when features are correlated
- No iterative optimisation needed — one matrix inversion

Bias-variance tradeoff:

- Large α : more shrinkage $\rightarrow$ higher bias, lower variance
- Small α : less shrinkage $\rightarrow$ lower bias, higher variance

Concrete Example: Why Ridge is Perfect for Reservoir Computing

In HPT-QRC the reservoir (photonic circuit) is *fixed* — we never train it. The only trainable component is the readout layer. Ridge solves this in one shot:

$$W_{\text{out}} = (\Phi^\top \Phi + \alpha I)^{-1} \Phi^\top y$$

where $\Phi \in \mathbb{R}^{T \times D}$ is the matrix of reservoir features. For $D = 270$ features and $T = 800$ training samples, this is a tiny (270×270) matrix inversion — solved in milliseconds.

Key Takeaway: Ridge has no barren-plateau problem, no gradient descent, no hyperparameter sensitivity beyond the single scalar α . It is the ideal readout for a fixed-reservoir architecture.

5 MSE and QLIKE Loss Functions

Two Ways to Measure Forecast Error

A loss function quantifies “how wrong” a forecast is. Different loss functions reward different properties.

Math: MSE — Mean Squared Error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{T} \sum_{t=1}^T (\hat{y}_t - y_t)^2$$

- Symmetric: overestimating and underestimating by the same amount are penalised equally
- Scale-dependent: a 0.001 error is treated the same whether $y = 0.001$ or $y = 1000$
- Sensitive to outliers (squaring magnifies large errors)

Math: QLIKE — Quasi-Likelihood Loss

$$\mathcal{L}_{\text{QLIKE}}(\hat{\sigma}^2, \sigma^2) = \frac{\sigma^2}{\hat{\sigma}^2} - \log \frac{\sigma^2}{\hat{\sigma}^2} - 1$$

- Always ≥ 0 ; equals 0 iff $\hat{\sigma}^2 = \sigma^2$
- **Scale-invariant**: penalises relative errors, not absolute ones
- **Asymmetric**: strongly penalises *underestimating* volatility (dangerous in risk management: you think it's calm when it isn't)
- **Patton (2011)**: QLIKE is *robust to noisy proxies* — using a noisy RV estimate instead of true variance still correctly ranks forecasting models

Concrete Example: Why QLIKE > MSE for Volatility

Suppose two forecasters predict next-month S&P 500 RV:

	True σ^2	Forecast $\hat{\sigma}^2$	MSE	QLIKE
Forecaster A	0.010	0.012	4×10^{-6}	0.003
Forecaster B	0.010	0.002	64×10^{-6}	1.609
Forecaster A	0.100	0.102	4×10^{-6}	0.000
Forecaster B	0.100	0.020	6400×10^{-6}	1.609

MSE treats Forecaster A the same at low and high volatility. QLIKE recognises that underestimating by a factor of 5 (Forecaster B) is catastrophic from a risk-management perspective.

Key Takeaway: In the paper, QLIKE is the *primary* metric for the S&P 500 task. All DM tests are run on both MSE and QLIKE. QLIKE is even reported on the synthetic benchmarks (NARMA-10, Mackey-Glass) for uniform comparability across datasets.

Part II

Machine Learning Foundations

6 Reservoir Computing and Echo State Networks (ESN)

Reservoir Computing — The Big Idea

Training a Recurrent Neural Network (RNN) is hard: gradients either *explode* or *vanish* as they flow through time (the vanishing-gradient problem). Reservoir Computing sidesteps this entirely by **never training the recurrent part at all**.

The idea:

1. Take a complex dynamical system (the “reservoir”) with random, *fixed* connections.
2. Drive it with your input time series.
3. The reservoir maps the history of inputs into a rich high-dimensional state space.
4. Train *only* a simple linear readout from reservoir states to outputs.

This is like having a pre-built pattern recogniser (the reservoir) and only learning how to *read* its outputs.

Math: Echo State Network (ESN) Equations

Reservoir state update:

$$\mathbf{x}_t = (1 - \alpha)\mathbf{x}_{t-1} + \alpha \cdot \tanh(W_{\text{in}} u_t + W_{\text{res}} \mathbf{x}_{t-1})$$

Readout (only trained part):

$$\hat{y}_t = W_{\text{out}} \mathbf{x}_t$$

- $\mathbf{x}_t \in \mathbb{R}^N$: reservoir state ($N \approx 200\text{--}1000$)
- $W_{\text{res}} \in \mathbb{R}^{N \times N}$: random sparse fixed matrix, spectral radius $\rho < 1$
- W_{in} : random fixed input matrix
- $\alpha \in (0, 1]$: leaking rate — controls speed of state update
- W_{out} : **only trained weights**, via Ridge regression

Echo State Property (ESP): With $\rho(W_{\text{res}}) < 1$, the reservoir “forgets” its initial state exponentially fast. The current state $\mathbf{x}_t$ depends only on the recent input history, not on arbitrary initial conditions.

Concrete Example: Why ESN is the Main Baseline in the Paper

ESN and HPT-QRC share the same structure:

	ESN	HPT-QRC
Reservoir type	Random RNN (classical)	Photonic interferometer
Reservoir training	None (fixed)	None (fixed)
Readout training	Ridge regression	Ridge regression
Memory source	Recurrent connections	Sliding window + Fock states

Comparing them is “apples-to-apples” for the reservoir computing paradigm.

Key Takeaway: ESN is swept over sizes $\{50, 100, 200, 500, 1000\}$ with Optuna tuning of leak rate, spectral radius, and ridge α . It is the most competitive classical reservoir baseline.

7 Random Fourier Features (RFF)

Kernel Methods Without the $O(T^2)$ Cost

Kernel methods (like SVMs) find nonlinear patterns by implicitly operating in a high-dimensional feature space via a kernel function $k(\mathbf{x}, \mathbf{y})$. The catch: computing k for all pairs of T training points costs $O(T^2)$ — too slow for large datasets.

Rahimi & Recht (NeurIPS 2007) had an elegant idea: instead of computing the kernel implicitly, *approximate it explicitly* using random projections. Their Random Fourier Features map inputs into a D -dimensional feature space where the dot product $\approx$ the kernel. Then just run linear regression on these features.

Math: RFF Construction (RBF Kernel)

For the Radial Basis Function (RBF) kernel $k(x, y) = e^{-\|x-y\|^2/2\sigma^2}$:

1. Sample D random frequencies: $\omega_j \sim \mathcal{N}(0, \sigma^{-2}I)$, and offsets $b_j \sim \mathcal{U}[0, 2\pi]$

2. Feature map:

$$\phi(\mathbf{x}) = \sqrt{\frac{2}{D}} [\cos(\omega_1^\top \mathbf{x} + b_1), \dots, \cos(\omega_D^\top \mathbf{x} + b_D)]$$

3. By Bochner’s theorem: $k(\mathbf{x}, \mathbf{y}) \approx \phi(\mathbf{x})^\top \phi(\mathbf{y})$

Then fit Ridge regression: $\hat{y} = \hat{\beta}^\top \phi(\mathbf{x})$.

Key Takeaway: In the paper, RFF+Ridge is the *matched-dimension kernel baseline* — tuned to the same feature dimension as HPT-QRC via Optuna. It tests whether HPT-QRC’s quantum Fock features are better than a carefully tuned classical kernel method of the same size.

8 Optuna and the TPE Algorithm

Hyperparameter Optimisation — Why it Matters

Every machine learning model has “dials” that must be set before training: learning rate, number of layers, regularisation strength, window size, etc. These are called *hyperparameters*. The right settings can be the difference between a model that works and one that doesn’t.

Grid search: try every combination — $O(k^d)$ evaluations for d hyperparameters, each with k values. Exponentially expensive.

Random search: try random combinations — better, but ignores what was learned from previous trials.

Bayesian optimisation (Optuna’s approach): model which hyperparameter regions are promising and *focus* evaluations there.

Math: TPE — Tree-structured Parzen Estimator

At each step, Optuna splits all previous trials into:

- $\ell(\lambda)$: density of “good” configs (top $\gamma = 25\%$ by loss)
- $g(\lambda)$: density of “bad” configs (bottom 75%)

Both are estimated as *Parzen windows* (mixture of Gaussians).

The next hyperparameter to try maximises:

$$\text{EI}(\lambda) \propto \frac{\ell(\lambda)}{g(\lambda)} \quad (\text{Expected Improvement})$$

Why TPE is practical:

- Handles continuous, integer, and categorical hyperparameters
- Handles conditional structure (e.g., “if model is LSTM, tune `n_layers`”)
- No matrix inversions — fast and scalable
- Default sampler in Optuna (the most popular HPO library)

Concrete Example: HPT-QRC Hyperparameter Search Space

Optuna tunes 5 hyperparameters for HPT-QRC:

Hyperparameter	Range	Best (Mackey-Glass)
<code>photon_list</code>	$\{[2], [3], [4], [2, 3], [2, 4], [3, 4], [2, 3, 4]\}$	$[2, 4]$
<code>window</code>	1–20	5
<code>n_virtual_nodes</code>	1–5	3
<code>lex_out</code>	4–20	14
<code>ridge_alpha</code>	10^{-8} – 10^{-1} (log scale)	10^{-6}

20 Optuna trials (fewer than 30 for classical baselines, since each HPT-QRC fit is slower).

Key Takeaway: Because HPT-QRC uses *fewer* trials (20 vs 30), any performance advantage is *conservative* — the quantum model is actually at a slight search disadvantage.

9 Walk-Forward Validation

Why Standard Cross-Validation is Dangerous for Time Series

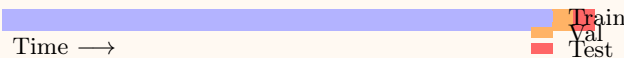
Standard k -fold cross-validation randomly shuffles your data and trains on some folds, tests on others. For i.i.d. data (e.g., image classification) this is fine. For time series it is **catastrophic**: if you train on data from 2020 and test on data from 2015, you are effectively “looking into the past from the future” — the model sees future information during training. This is called *look-ahead bias* and produces overly optimistic results.

Walk-forward validation fixes this by always respecting the arrow of time: **train on the past, test on the future**.

Math: Walk-Forward Protocol in the Paper

For S&P 500 RV (1950–2017, 68 years of monthly data):

- Train window: 20 years
- Validation window: 2 years (for hyperparameter selection)
- Test window: 2 years
- Slide: 2 years forward after each fold
- Total folds: 8 (covering 1970–2017)



Results are reported as *median* and IQR across the 8 test folds.

Key Takeaway: Walk-forward is harder than a fixed 80/20 split. HPT-QRC (default config) trails HAR in walk-forward median MSE. The paper is transparent about this — the tuned walk-forward result is deferred to the journal version.

Part III

Statistical Inference

10 The Diebold-Mariano (DM) Test

Are Two Forecasters Really Different, or Just Lucky?

Suppose Model A has lower MSE than Model B on a test set of 100 months. Is A genuinely better, or did it just happen to be luckier on these particular 100 months? The Diebold-Mariano (DM) test answers this question rigorously using hypothesis testing. Think of it as the forecasting equivalent of a two-sample t-test.

Math: DM Test Step-by-Step

Step 1. Compute loss differential at each time point:

$$d_t = L(\hat{y}_t^A, y_t) - L(\hat{y}_t^B, y_t)$$

e.g., $d_t = (\hat{y}_t^A - y_t)^2 - (\hat{y}_t^B - y_t)^2$ for MSE.

Step 2. Null hypothesis:

$$H_0 : \mathbb{E}[d_t] = 0 \quad (\text{equal expected accuracy})$$

Step 3. Test statistic:

$$DM = \frac{\bar{d}}{\sqrt{\widehat{\text{Var}}(\bar{d})}} \xrightarrow{d} \mathcal{N}(0, 1)$$

where $\bar{d} = \frac{1}{T} \sum_t d_t$ and $\widehat{\text{Var}}(\bar{d})$ is a HAC variance estimate (see Section 11).

Step 4. For a two-sided test at 5% level: reject H_0 if $|DM| > 1.96$.

Interpretation:

- $DM > 0$ (and significant): Model B is better (A has higher loss)
- $DM < 0$ (and significant): Model A is better (B has higher loss)
- $|DM| < 1.96$: cannot tell them apart at 5% level

Concrete Example: Reading Table 5 in the Paper

For HPT-QRC (tuned) vs. HAR on S&P 500 RV:

Metric	DM stat	p-value
MSE	1.979	0.048
QLIKE	1.732	0.083

$DM = 1.979 > 0$ means HAR has lower MSE than HPT-QRC. $p = 0.048 < 0.05$: statistically significant at 5%.

Translation: “HAR beats HPT-QRC on MSE at the 5% significance level. However, on QLIKE ($p = 0.083 > 0.05$), we cannot distinguish them.” This is consistent with HPT-QRC being “in the same statistical class” as HAR — it doesn’t clearly beat HAR, but also isn’t clearly worse on the volatility-appropriate loss.

Key Takeaway: The DM test is the core of the paper’s statistical claims. Without it, saying “Model A has lower error than Model B” is just descriptive — it could be noise. The DM test says whether the difference is real.

11 Newey-West HAC Variance

Why Regular Standard Errors are Wrong for Forecast Errors

In a standard regression, error terms are assumed to be independent (i.i.d.). But forecast errors are *not* independent: if you underestimate volatility today, you’ll likely underestimate it tomorrow too (serial correlation). Also, errors vary in size over time (heteroskedasticity — think calm vs. volatile market periods).

Using standard error formulas under these violations *underestimates* the true variance of $\bar{d}$, making the DM statistic look larger (more significant) than it really is — you’d get spurious p-values.

HAC stands for **H**eteroskedasticity and **A**utocorrelation **C**onsistent. The Newey-West estimator corrects for both problems.

Math: Newey-West (1987) HAC Estimator

$$\hat{V}_{\text{HAC}} = \hat{\Gamma}_0 + \sum_{j=1}^m w_j (\hat{\Gamma}_j + \hat{\Gamma}_j^{\top})$$

where the j -th autocovariance is:

$$\hat{\Gamma}_j = \frac{1}{T} \sum_{t=j+1}^T d_t \cdot d_{t-j}$$

and Bartlett kernel weights are:

$$w_j = 1 - \frac{j}{m+1}, \quad j = 1, \dots, m$$

The bandwidth m controls how many lags of autocorrelation to account for.

Math: Andrews (1991) Automatic Bandwidth

How large should m be? Too small: miss important autocorrelations. Too large: add noisy lag estimates. Andrews (1991) derived the asymptotically optimal data-driven choice:

$$m^* = \left\lfloor 4 \left(\frac{T}{100} \right)^{2/9} \right\rfloor$$

For $T = 135$ monthly observations: $m^* = \lfloor 4 \times (1.35)^{2/9} \rfloor = \lfloor 4 \times 1.068 \rfloor = 4$.

This formula comes from minimising the asymptotic mean squared error of the HAC estimator under the Bartlett kernel.

Key Takeaway: Every DM p -value in the paper uses HAC standard errors with the Andrews automatic bandwidth. This is the *econometrically correct* way to run DM tests — most ML papers skip this step and get over-confident p -values.

12 Hansen Model Confidence Set (MCS)

The Multiple Comparisons Problem

If you compare 10 models pairwise with 5% tests, you'd run $\binom{10}{2} = 45$ tests. Even if all models are identical, you expect $45 \times 0.05 \approx 2$ false positives. This is the *multiple comparisons problem* — you can't just run many DM tests and collect “significant” results.

Hansen's Model Confidence Set (2011) solves this by producing a *set of models* that cannot be distinguished from the best model at a given confidence level — a “confidence interval for the identity of the best model.”

Math: MCS Algorithm

Input: Set of models $\mathcal{M}_0$, confidence level $\alpha = 0.10$.

Algorithm:

1. Test $H_0^{\mathcal{M}}$: all models in $\mathcal{M}$ have equal expected loss (using stationary bootstrap for p -values)
2. If H_0 *not rejected*: stop. Current $\mathcal{M}$ is the MCS.
3. If H_0 *rejected*: remove the worst model (highest average loss differential vs. others) from $\mathcal{M}$, go to step 1.

Output: $\hat{\mathcal{M}}_{1-\alpha}^*$ — the 90% MCS survivor set.

Interpretation: Models in the MCS are statistically indistinguishable from the best model. Being in the MCS is the forecasting analogue of “not significantly worse than the winner.”

Concrete Example: MCS Results in the Paper

For S&P 500 RV (fixed split), the MCS retains *all* 14 models with $p = 0.39$ — with only $T \approx 135$ test months, the data does not have enough power to separate any model from the best. This is the standard finding in the RV forecasting literature: HAR and its competitors are hard to distinguish at monthly frequency with real data.

For walk-forward (8 folds, $T = 192$ test points combined): all 8 models in the MCS (MSE: $p = 0.125$, QLIKE: $p = 0.312$). Same conclusion.

Stationary Bootstrap. The MCS uses bootstrap to generate critical values. The *stationary bootstrap* (Politis & Romano 1994) resamples blocks of data with *geometrically distributed random lengths* (mean block length = 20 in the paper), preserving the autocorrelation structure of the loss differential series. The paper uses $B = 2000$ bootstrap resamples.

Key Takeaway: The MCS + DM combination is the gold standard for forecast comparison in econometrics. The paper uses it correctly — this is what makes it publishable in econometrics venues.

Part IV

Quantum Photonics

13 Quantum States and Fock Space

Particles That Are All Identical — Bosons

In everyday life, objects are distinguishable: “the red ball” vs. “the blue ball.” Quantum particles are different. *Bosons* (like photons, the particles of light) are **fundamentally indistinguishable** — there is no “photon number 1” vs. “photon number 2.” The only meaningful question is: “how many photons are in each location (mode)?”

An optical “mode” is a specific spatial/temporal/polarisation channel that a photon can occupy. Think of modes as slots in a box.

Math: Fock States

A *Fock state* specifies the occupation number of each mode:

$$|n_1, n_2, \dots, n_m\rangle$$

n_i = number of photons in mode i .

Example: $|1, 0, 2, 0\rangle$ means: mode 1 has 1 photon, mode 3 has 2 photons, modes 2 and 4 are empty. Total: 3 photons in 4 modes.

Counting Fock states with exactly n photons in m modes:

$$\text{Full Fock space : } \binom{n+m-1}{n} \quad (\text{stars-and-bars})$$

$$\text{Unbunched (max 1 per mode) : } \binom{m}{n}$$

Concrete sizes:

n	m	Full	Unbunched	HPT-QRC
2	6	21	15	$\binom{6}{2} = 15$
3	6	56	20	$\binom{6}{3} = 20$
4	6	126	15	$\binom{6}{4} = 15$

Key Takeaway: Fock states are the “vocabulary” of photonic computing. The HPT-QRC paper extracts the *probability* of each Fock state at the output of the interferometer and uses these probabilities as features for regression.

14 Linear Optical Interferometers and Phase Encoding

What is a Photonic Interferometer?

An optical interferometer is a network of beam splitters and phase shifters that redirects and mixes light beams. Mathematically it implements a *unitary transformation* U on the photon mode operators.

Think of it as a “photon mixer”: photons entering from the left emerge at the right with transformed probabilities, governed by interference.

Math: Building Blocks

Beam Splitter (BS): Mixes two modes with reflectivity θ :

$$\begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} \cos \theta & i \sin \theta \\ i \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$$

A 50:50 BS ($\theta = \pi/4$) sends half the photons to each output.

Phase Shifter (PS): Multiplies a mode by a phase:

$$b = e^{i\phi} a$$

This is the **encoding mechanism** in HPT-QRC: the time-series input $x_t \in [0, 1]$ is encoded as $\phi = 2\pi x_t$.

Mach-Zehnder Interferometer (MZI): Two 50:50 beam splitters with a phase shifter between them — a tunable beam splitter with continuously adjustable splitting ratio. Cascading MZIs can implement *any* unitary transformation.

How Input Is Encoded into the Circuit

Step 1. Take the time-series window $\mathbf{x}_t = [y_{t-W+1}, \dots, y_t]$ (e.g., 10 past values).

Step 2. Winsorise and min-max scale to $[0, 1]$.

Step 3. Each value controls one input phase shifter: $\phi_k = 2\pi x_{t-W+k}$.

Step 4. The Fock input state $|1^n 0^{m-n}\rangle$ (n photons in the first n modes, rest empty) enters the phase-encoded circuit.

Step 5. The circuit’s fixed random beam splitter network (the “reservoir”) mixes these phase-encoded modes through quantum interference.

Step 6. Measure the output Fock probabilities — these are the features.

Key Takeaway: The fixed random interferometer parameters are the “reservoir” — they are never trained. Only the input phase encoding changes with each time step. This is analogous to how ESN reservoir weights are random and fixed, but input weights map the current input to the reservoir.

15 The Permanent of a Matrix and Boson Sampling

Why Photon Counting is Hard to Simulate Classically

When n indistinguishable photons pass through an m -mode interferometer, the output probabilities involve a mathematical quantity called the *permanent* of a matrix. Computing the permanent is among the hardest problems in mathematics.

Math: Permanent vs. Determinant

For an $n \times n$ matrix A :

$$\det(A) = \sum_{\sigma \in S_n} \underbrace{\text{sgn}(\sigma)}_{\pm 1} \prod_{i=1}^n a_{i,\sigma(i)} \quad (\text{computable in } O(n^3))$$

$$\text{Perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i,\sigma(i)} \quad (\text{no sign! computable in } O(2^n n))$$

The *only* difference is the absence of the sign $\text{sgn}(\sigma)$. Yet this makes the permanent **#P-hard** to compute (Valiant 1979) — believed to be harder than NP.

Fock state transition probability:

$$P(|S\rangle \rightarrow |T\rangle) = \frac{|\text{Perm}(U_{ST})|^2}{\prod_i s_i! \prod_j t_j!}$$

where U_{ST} is the submatrix of the interferometer unitary U formed by selecting columns per input state S and rows per output state T .

Boson Sampling (Aaronson-Arkhipov 2011)

Task: Sample from the output Fock distribution of a random m -mode interferometer with $n \ll m$ photons.

Computational claim: No polynomial-time classical algorithm exists for this (unless the polynomial hierarchy collapses).

Why it matters for HPT-QRC: The features extracted by HPT-QRC come from this distribution. In principle, they represent a classically hard-to-reproduce probability distribution.

Note: The paper explicitly does **NOT** claim quantum advantage for regression — only that Fock-space features are expressive and useful.

16 Perceval, SLOS, and MerLin

Perceval — The Photonic Simulation Engine

Perceval is Quandela's open-source Python library for simulating photonic quantum circuits. Think of it as PyTorch for photonic computing.

Key idea: You describe a circuit (beam splitters, phase shifters, photon sources, detectors). Perceval simulates what happens to the photons and returns output probabilities.

Math: SLOS Backend — Scalable Linear Optical Simulator

The default exact simulation backend. For n photons in m modes:

Algorithm:

1. Start with input Fock state $|n_1, \dots, n_m\rangle$
2. Propagate amplitude column-by-column through the circuit, contracting the bosonic Hilbert space at each step
3. The result: a full output probability distribution over all valid Fock states

Complexity: $O\left(\binom{n+m-1}{n} \cdot m\right)$ — polynomial in m for fixed n , but exponential in n .

Why SLOS is exact: Unlike approximations (neural-network surrogate models, sampling methods), SLOS computes exact permanents, giving exact Fock probabilities. This is crucial for HPT-QRC — we need deterministic, noise-free features from the simulation.

ComputationSpace.UNBUNCHED — At Most One Photon Per Mode

When we restrict to the **unbunched** subspace, we only consider Fock states where every mode has 0 or 1 photon ($n_i \in \{0, 1\}$).

Why unbunched:

- Smaller feature space: $\binom{m}{n}$ instead of $\binom{n+m-1}{n}$
- Easier to implement experimentally: coincidence detection (one photon per detector at a time) is standard hardware
- On real hardware (Ascella), bunched events are identified by photon-number-resolving detectors and *discarded* via post-selection, recovering the unbunched distribution

MerLin — Differentiable Photonic ML on Perceval

MerLin wraps Perceval circuits into PyTorch `nn.Module` objects, enabling integration with standard deep learning pipelines.

In HPT-QRC, the key MerLin component is **QuantumLayer**: a fixed photonic circuit whose forward pass returns Fock output probabilities. No gradients flow through the circuit (reservoir is frozen) — only the Ridge readout is trained.

17 LexGrouping — Compressing Fock Probabilities

The Dimensionality Problem

After SLOS simulation in the unbunched subspace, we have $\binom{m}{n}$ output probabilities. For the heterogeneous ensemble $[2, 3, 4]$ photons with $m = 6$ modes:

$$\binom{6}{2} + \binom{6}{3} + \binom{6}{4} = 15 + 20 + 15 = 50 \text{ features per virtual layer}$$

With 3 virtual layers: 150 features. But we want to control this flexibly. LexGrouping does this compression.

Math: LexGrouping Algorithm

Given $K = \binom{m}{n}$ Fock probabilities $p_1, \dots, p_K$ (indexed in lexicographic order of Fock states):

Step 1. Pad to $K' = \lceil K/D \rceil \cdot D$ by appending zeros.

Step 2. Divide into D contiguous groups of $\lceil K/D \rceil$ each.

Step 3. Sum within each group:

$$[\text{LexGroup}(\mathbf{p})]_d = \sum_{k=(d-1)\lceil K/D \rceil}^{d\lceil K/D \rceil-1} p_k, \quad d = 1, \dots, D$$

Result: A D -dimensional vector. Total probability is preserved ($\sum_d [\text{LexGroup}]_d = \sum_k p_k$). D is the tunable hyperparameter `lex_out`.

Concrete Example: LexGrouping with 6 Fock States into 3 Groups

Unbunched states for $n = 2, m = 4$: $|1100\rangle, |1010\rangle, |1001\rangle, |0110\rangle, |0101\rangle, |0011\rangle$ with probabilities $[0.3, 0.1, 0.05, 0.2, 0.15, 0.2]$.

LexGroup into $D = 3$ groups (2 states per group):

$$\begin{aligned} g_1 &= 0.3 + 0.1 = 0.40 \\ g_2 &= 0.05 + 0.2 = 0.25 \\ g_3 &= 0.15 + 0.2 = 0.35 \end{aligned}$$

Output: $[0.40, 0.25, 0.35]$ — sums to 1.0. ✓

Key Takeaway: LexGrouping is a parameter-free, deterministic compression that preserves total probability. It is not a learned layer — no training required. The tunable parameter is simply D (`lex_out`), which controls feature dimension.

Part V

Benchmarks and Architecture

18 NARMA-10 Benchmark

A Nonlinear Memory Challenge

NARMA stands for Nonlinear AutoRegressive Moving Average. NARMA-10 is a synthetic time series that requires a model to:

- Integrate information from the last 10 time steps (memory)
- Combine past outputs and past inputs *nonlinearly*
- No purely linear method can solve it optimally

It is the most widely used synthetic benchmark in reservoir computing literature.

Math: Exact NARMA-10 Formula

$$y_{t+1} = 0.3 y_t + 0.05 y_t \sum_{i=0}^9 y_{t-i} + 1.5 u_{t-9} u_t + 0.1$$

where $u_t \sim \mathcal{U}[0, 0.5]$ is the driving input.

Breakdown:

- $0.3 y_t$: linear AR(1) component
- $0.05 y_t \sum_{i=0}^9 y_{t-i}$: nonlinear term coupling current output to sum of 10 past outputs
- $1.5 u_{t-9} u_t$: input product term — requires memory of at least 9 steps

Metric: Normalised MSE (NMSE) = $\text{MSE}/\text{Var}(y)$. NMSE = 1.0 means the model is no better than predicting the mean.

Concrete Example: Why NARMA-10 Favours HPT-QRC

The IPC analysis in the paper shows that HPT-QRC achieves $\approx 1.3\times$ more degree-3 nonlinear capacity than ESN at matched feature dimension. The nonlinear terms in NARMA-10 (especially the product $u_{t-9} \cdot u_t$) require exactly this cubic-order processing. This is why HPT-QRC-X achieves 3.98×10^{-4} MSE vs. RFF+Ridge-X's 4.36×10^{-4} .

19 Mackey-Glass Benchmark

A Classic Chaotic System

The Mackey-Glass equation was originally proposed in 1977 as a model of blood cell regulation with time-delayed feedback. For the right parameter setting, it produces *chaotic* dynamics: the system is entirely deterministic but its long-run behaviour is unpredictable (sensitive to initial conditions).

It is the standard “hard” time-series benchmark in reservoir computing.

Math: Mackey-Glass Equation

$$\frac{dx}{dt} = \frac{\beta x(t - \tau)}{1 + x(t - \tau)^n} - \gamma x(t)$$

Standard parameters: $\beta = 0.2$, $\gamma = 0.1$, $n = 10$.

Chaos parameter τ :

- $\tau < 17$: periodic oscillations (not chaotic)
- $\tau = 17$: mildly chaotic — used in most RC papers
- $\tau = 30$: strongly chaotic
- $\tau \geq 100$: hyperchaos

For $\tau = 17$, the system has a positive Lyapunov exponent $\lambda \approx 0.006$ — nearby trajectories separate exponentially, but slowly enough that short-horizon prediction is feasible.

In the paper: Integrate with Runge-Kutta-4, subsample every 10 steps, predict x_{t+1} from x_t . Task: one-step-ahead forecasting.

Key Takeaway: Tuned HPT-QRC achieves 3.3×10^{-7} MSE on Mackey-Glass ($\tau = 17$) — approximately $4\times$ better than the best classical baseline (RFF+Ridge at 1.32×10^{-6}). This is the paper’s strongest quantitative result.

20 Information Processing Capacity (IPC)

Measuring What a Reservoir Can “Remember” and “Compute”

The Information Processing Capacity (IPC) framework (Dambre et al. 2012) provides a principled way to characterise what a reservoir is good at. The idea: any reservoir with N state dimensions can, at most, process N independent computational tasks (a conservation law for information).

IPC decomposes this total budget into:

- **Linear MC:** how well can the reservoir reconstruct *linear functions* of past inputs?
- **IPC_{deg 1}:** degree-1 (linear) processing

- $\text{IPC}_{\text{deg } 2}$: degree-2 (quadratic) processing
- $\text{IPC}_{\text{deg } 3}$: degree-3 (cubic) processing

Math: IPC Definition

For a target function f of past inputs (e.g., $f = u_{t-3}^2$ or $f = u_{t-1} \cdot u_{t-5}$):

$$C_f = \sup_{W_{\text{out}}} \frac{\text{Cov}^2(\hat{y}_t, f(\mathbf{u}_t))}{\text{Var}(\hat{y}_t) \cdot \text{Var}(f(\mathbf{u}_t))}$$

This is the squared correlation (R^2) of the best linear readout's ability to reconstruct f from the reservoir state.

Total IPC Theorem (Dambre 2012):

$$\sum_{f \in \mathcal{F}} C_f \leq N$$

Sum over orthonormal basis functions; bounded by reservoir dimension N .

Concrete Example: IPC Results in the Paper (Table 7)

At matched feature dimension ≈ 95 :

System	Linear MC	$\text{IPC}_{\text{deg } 1}$	$\text{IPC}_{\text{deg } 2}$	$\text{IPC}_{\text{deg } 3}$
HPT-QRC-Hetero [2, 3, 4]	5.00	5.00	14.90	32.62
ESN res=200 (tuned)	17.85	5.99	20.38	24.55
Random-linear (dim 100)	5.01	5.00	0.00	3.62

Reading:

- ESN has much higher *linear* MC (17.85 vs. 5.00) — its recurrent connections create a linear memory buffer
- HPT-QRC has higher *degree-3* IPC (32.62 vs. 24.55) — multi-photon interference creates cubic nonlinear features that ESN cannot replicate at this size
- This explains why HPT-QRC outperforms ESN specifically on NARMA-10 and Mackey-Glass — both require cubic nonlinear processing

21 Virtual Depth

More Layers = More Features, No Training Required

A single interferometer pass takes your 10-dimensional input window and outputs, say, 10 Fock probabilities. That is a relatively small feature vector. *Virtual depth* (also called “virtual nodes” in the RC literature) stacks multiple independent reservoir layers — each with a different random seed — to multiply the feature count. Crucially, no additional training is required — each layer is just another fixed random interferometer.

Math: Feature Dimension with Virtual Depth

Single layer: features = $|\text{photon_list}| \times \text{lex_out}$

With virtual depth V : features = $|\text{photon_list}| \times V \times \text{lex_out}$

Concrete example (default config): $[2, 3, 4]$ (3 photon numbers) $\times 3$ (virtual depth) $\times 10$ (lex-out) = 90 quantum features, plus 5 for the raw window context = 95 total.

Optuna-tuned config for Mackey-Glass: $[2, 4]$ (2 photon numbers) $\times 3$ (virtual depth) $\times 14$ (lex-out) = 84 quantum features.

22 The Fock-Space Scaling and Heterogeneous Ensemble

Why Combine Multiple Photon Numbers?

Different photon numbers n correspond to different Fock subspaces, each with different dimension $\binom{m}{n}$ and different multi-photon interference statistics. The key insight of HPT-QRC:

- $n = 2$: 2-photon interference (Hong-Ou-Mandel-like) — quadratic features
- $n = 3$: 3-photon interference — cubic features (highest IPC degree-3)
- $n = 4$: 4-photon interference — quartic features

Combining these in a *heterogeneous ensemble* captures feature diversity across multiple polynomial orders simultaneously. This is the core architectural innovation of the paper.

Math: Fock Dimension is Not Monotone in Photon Number

For $m = 8$ modes:

n photons	$\binom{8}{n}$ (unbunched)	Fock dim
1	$\binom{8}{1} = 8$	8
2	$\binom{8}{2} = 28$	28
3	$\binom{8}{3} = 56$	56
4	$\binom{8}{4} = 70$	70
5	$\binom{8}{5} = 56$	56

Note: $\binom{8}{3} = \binom{8}{5} = 56$ — the dimension is *symmetric* around $n = m/2$ and is *not monotone*. This is why the paper plots NRMSE against Fock dimension rather than photon number.

Quick-Reference Glossary

Term	One-line definition
AR(p)	Predict y_t from last p values of y
HAR	AR with daily, weekly, monthly averages; 4 params; hard to beat
HARX	HAR + exogenous regressors
Realized Volatility	Sum of squared intraday returns; observable proxy for variance
QLIKE	Quasi-likelihood loss for variance; scale-invariant, asymmetric
Ridge	OLS + L_2 penalty; closed-form; essential for large feature spaces
ESN	Random-fixed recurrent reservoir + trained linear readout
RFF	Random cosine features that approximate a kernel; classical baseline
Optuna/TPE	Bayesian hyperparameter optimisation; focuses search on promising regions
Walk-forward	Time-series CV that respects time ordering; no look-ahead bias
DM test	Hypothesis test: are two forecasters statistically different?
HAC variance	Corrects standard errors for serial correlation + heteroskedasticity
Andrews band-width	Automatic optimal lag count for HAC estimator
MCS	Set of models statistically indistinguishable from the best
Fock state	Quantum state $ n_1, \dots, n_m\rangle$ specifying photon counts per mode
Interferometer	Network of beam splitters + phase shifters; implements unitary U
Phase encoding	Map time-series values to phase shifts $\phi = 2\pi x$
Permanent	Matrix function = determinant without signs; #P-hard; governs photon probabilities
SLOS	Perceval's exact photonic simulator; computes permanents exactly
UNBUNCHED	Restrict to at-most-one-photon-per-mode; $\binom{m}{n}$ states
LexGrouping	Compress $\binom{m}{n}$ Fock probs into D sums of contiguous groups
Virtual depth	Stack V independent random reservoirs; multiply features by V
NARMA-10	Nonlinear benchmark; requires 10-step memory + cubic interactions
Mackey-Glass	Chaotic delay-differential equation; $\tau = 17$ mild chaos
IPC	How many independent computational tasks can a reservoir perform?
Barren plateau	Vanishing gradients in quantum ML; avoided by fixed reservoirs
Boson sampling	Hard-to-simulate photon distribution; source of HPT-QRC expressivity
MerLin	PyTorch wrapper around Perceval; provides <code>QuantumLayer</code>
Perceval	Quandela's photonic circuit simulator (Python)
Ascella	Quandela's 12-mode photonic quantum processor
Winsorize	Clip outliers at 1st/99th percentile (robust preprocessing)
IQR scaling	Normalise by quartile range; robust to outliers
Sliding window	Convert scalar series to fixed vectors: $[y_{t-W+1}, \dots, y_t]$

Reading Order Recommendation

If you are starting from scratch, read the concepts in this order:

1. **Time series basics:** $\text{AR}(p) \rightarrow \text{HAR} \rightarrow \text{Realized Volatility}$
2. **Loss functions:** $\text{MSE} \rightarrow \text{QLIKE}$ (understand *why* QLIKE exists)
3. **The readout:** Ridge Regression (the only trained component)
4. **Classical reservoir:** Reservoir Computing $\rightarrow$ ESN (the classical analogue of HPT-QRC)
5. **Quantum basics:** Fock states $\rightarrow$ Interferometers $\rightarrow$ Phase encoding $\rightarrow$ Permanent $\rightarrow$ SLOS $\rightarrow$ UNBUNCHED $\rightarrow$ LexGrouping
6. **Architecture:** Virtual depth $\rightarrow$ Heterogeneous ensemble $\rightarrow$ NARMA-10 $\rightarrow$ Mackey-Glass $\rightarrow$ IPC
7. **Statistical tests:** DM test $\rightarrow$ HAC $\rightarrow$ Andrews bandwidth $\rightarrow$ MCS
8. **Hyperparameter tuning:** Walk-forward $\rightarrow$ Optuna/TPE

Once you understand steps 1–4 (classical) and 5–6 (quantum), the paper reads naturally: it is a story of “can quantum Fock-space features replace classical reservoir features?” The answer: yes for chaotic benchmarks (IPC degree-3 advantage), approximately-yes for financial volatility (statistically indistinguishable from HAR in fixed-split, trails in walk-forward).